

통합반 8회 · Process GPT에 업무 Skill 적용

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 사전 등록된 Skill을 업무 에이전트에 할당 · 50분](#)
- [4. 활동 2 — 제공 도구 목록에서 상태·검색·조치 도구 확인 · 50분](#)
- [5. 활동 3 — 프로세스 템플릿에 이벤트 입력 매핑 · 50분](#)
- [6. 활동 4 — 사람 승인과 거부 결과를 업무 화면에서 확인 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [10. 실제 플랫폼 대응](#)
- [강사 노트](#)
- [용어](#)

날짜·시간	2027-02-02 (화) 09:00~13:00	모듈	M3 · 원 일정 19번
블록	확장 B6, B7, B8 / 플랫폼 B6, B7, B8	산출물	플랫폼 업무 인스턴스와 승인 기록
완료 확인	학생별 새 에이전트 프레임워크 개발 없이 기능 연결		

회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · 확장 조건·연결을 바꾼다 · 플랫폼 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질·정칙 수정과 검증)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 10/31	구현	구현	구현						
2회 11/07		확장	확장						
3회 11/21			확장	구현					
4회 11/28		확장	확장		구현			구현	
5회 01/25				확장		구현	구현		
6회 01/26			플랫폼	확장+P			확장		
7회 01/27						확장+P	확장+P	구현+P	
8회 01/28	확장				확장			확장	
9회 01/29									확장+P

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 11/14	구현	구현							
2회 01/25		확장	구현						
3회 01/26				구현					
4회 01/27		확장	확장		구현			구현	
5회 01/28				확장		구현			
6회 01/29							구현	구현	
7회 02/01			플랫폼	확장+P					
8회 02/02						확장+P	확장+P	확장+P	
9회 02/03									확장+P
10회 02/04	확장				확장			확장	확장

이번 회차에서 할 일

5회에 채운 SKILL.md(B6)와 6회에 연결한 도구·승인 흐름(B7→B8)을 이번에는 **교육용 플랫폼(Lab Platform)** 의 Agents 계층과 Orchestration 계층에 붙인다. 에이전트를 새로 만들지 않는다. 이미 등록된 Skill 과 도구 서버를 업무 에이전트에 **설정으로 할당**하고, 제공 프로세스 템플릿의 입력 폼에 사건 값이 어떻게 흘러가는지 빈칸을 채운다. 끝나면 사건 하나가 "접수 → 근거 확인·제안 → 사람 승인 → 조치 → 재점검" 업무 인스턴스로 진행되고, 승인과 거부가 각각 다른 기록으로 남는 것을 업무 화면에서 확인할 수 있다.

학습 목표

- 업무 에이전트 설정(`skills` , `mcp_servers` , `allowed_tools`)을 채우고, 실행 도구를 허용 목록에서 빼는 이유를 설명할 수 있다.
- 도구 6종을 상태 조회·검색·제안 검사·실행·재측정으로 나눌 수 있다.
- 프로세스 템플릿에서 `event_id` · `asset_id` · `run_id` · 제안값·근거 참조가 승인 폼까지 이어지는 매핑(`${...}`)을 채울 수 있다.
- 승인 결과와 거부 결과를 인스턴스 단계 기록과 사건 기록에서 찾아 비교할 수 있다.

1. 시작 전 준비 (복습·환경 확인)

```
cd lecture/system
curl -s http://localhost:8910/agents/skills
curl -s http://localhost:8910/agents/mcp-servers
```

예상 출력 (2026-09-14 실행):

```
[{"name":"hydraulic-cooling-response","chars":1614,"version":1,"description":"유압 설비 냉각 이상·센서 오류 사건에 대해 근거 있는 조치 제안을 만드는 업무 지침 (교육용)", ...}]
[{"url":"http://localhost:8800/mcp/","name":"hydops","type":"http", ...}]
```

Skill 하나와 도구 서버 하나가 강사 준비로 이미 등록돼 있다. `chars 1614` 는 frontmatter 를 뺀 본문 길이이다.

복습 질문: 6회에 "도구와 Skill 이 다른 역할"이라고 정리했다. 아래 표의 빈칸을 짝과 채워 본다.

질문	Skill(SKILL.md)	SOP	도구
무엇을 어떤 순서로 판단하나	✓		
무엇이 허용되고 누가 승인하나		✓	
실제로 조회·실행·검사하나			✓

2. 핵심 개념

Skill 은 업무 지침, SOP 는 근거, 도구는 실행 기능

SKILL.md 에 승인 규칙을 썼다는 것만으로 승인 검사가 구현되지는 않는다. 검사는 도구의 코드가 한다.

Skill · SKILL.md

무엇을 어떤 순서로 판단하나

- 센서 품질 확인 → 대상 설비 → SOP 검색 → 제안 → 승인 확인 → 실행 → 재측정
- 인용 규칙: 검색 결과에 실제로 나온 절만
- 중단 규칙: 근거 없으면 HOLD, 센서 오류면 SENSOR_CHECK
- 짧게(수업용 16천 자) · 에이전트가 실행 시 로드

SOP · 문서 근거

무엇이 허용되고 누가 승인하나

- 5중: 센서 오류 점검 · 냉각 이상 확인 · 승인 부하 감소 · 조치 후 재점검 · 미개선 이관
- 같은 양식: 적용 설비 · 발동 조건 · 확인할 관측 · 허용 조치 · 승인 주체 · 재점검 기준
- 문서 ID 버전-유효일 → 긴 본문은 검색 도구로 불러온다

도구 · 실행 기능

실제로 조회하고 실행하고 검사한다

- DB-그래프 조회, 시뮬레이터 조치, 재측정 편집
- 허용 조사·값 범위·승인 기록·사건 유효성·종류 키키를 코드에서 확인
- LangChain 도구 · MCP 서버 · 플랫폼 프로세스가 같은 함수를 공유

도구	입력	결과와 제한	누가 호출하나
get_asset_context	asset_id	센서 · 적용 SOP · 허용 조치(값 범위)	에이전트
get_recent_window	asset_id, 기간	관측 요약 · 단위 · 품질 플러그 — 원시 배열은 주지 않는다	에이전트
search_sop	asset_id, event_type	문서 ID · 버전 · 절 · 근거 본문 (설비·active 필터 + 그래프 확인)	에이전트
propose_action	event_id, 근거 참조	제안과 허용 범위 검사 결과 — 실행하지 않는다	에이전트
execute_sim_action	event_id, 승인 ID, 조치값	승인 테이블 기록을 검사한 뒤 실행하고 결과 기록	오케스트레이션(승인 후)
verify_recovery	조치 ID, 재측정 기간	회복 · 미개선 · 데이터 부족	오케스트레이션

Skill 은 업무 지침, SOP 는 근거, 도구는 실행 기능

구성	교육용 플랫폼에서의 모습	코드
Skill	SKILL.md 파일 하나. 에이전트 실행 시 시스템 프롬프트 "[할당된 스킬 가이드]" 아래에 붙는다(최대 6,000자)	labplatform/agents.py system_prompt
MCP 도구 서버	type: http 와 URL 만 받는다. 명령 기반 설정은 400으로 거부한다	register_mcp
업무 에이전트	agent_id + 할당 Skill + 도구 서버 + 허용 도구 목록	upsert_agent , run_agent_async
프로세스 정의	활동(폼·시스템·에이전트·사람·도구·종료)과 전이 조건	labplatform/process.py , templates/process_cooling_response.json

구성	교육용 플랫폼에서의 모습	코드
워크리스트	사람 태스크(조치 승인)가 기다리는 목록. 한 번 제출하면 끝	<code>worklist</code> , <code>complete</code>
Watch Agent	SQL 한 건 → 조건 <code>rows > 0</code> → 업무 시작	<code>watch_tick</code>

에이전트가 실행되면 도구 서버에서 도구를 모두 받아 온 뒤 허용 목록에 있는 도구만 남긴다.

```
# labplatform/agents.py - run_agent_async
tools = await _client(ag["mcp_servers"]).get_tools()
if ag["allowed_tools"]:
    tools = [t for t in tools if t.name in ag["allowed_tools"]]
agent = create_agent(model=ChatOpenAI(model=ag["model"], temperature=0),
                    tools=tools,
                    system_prompt=system_prompt(ag), response_format=Proposal)
```

주의 — SKILL.md 에 "승인 전에는 실행하지 않는다"라고 썼다고 승인 검사가 생기지 않는다. 승인 검사는 조치 도구 `execute_sim_action` 의 코드가 승인 테이블 기록으로 한다. 그래서 업무 에이전트에는 실행 도구를 아예 주지 않고, 실행은 사람 승인 뒤 프로세스의 도구 활동이 한다.

주의 — 프로세스 완료는 설비 회복이 아니다. 인스턴스 `status: COMPLETED` 는 업무가 끝났다는 뜻이다. 설비가 회복됐는지는 따로 저장된 `equipment_outcome` (재측정 판정)으로 읽는다.

3. 활동 1 — 사전 등록된 Skill을 업무 에이전트에 할당 · 50분

목표

`labs/integrated/S08/agent_assignment.json` 의 빈칸을 채우고 업무 에이전트 `ops-agent` 에 Skill 을 할당한다.

따라 하기

1. 콘솔 `http://localhost:8910/console/` → **Agents** → Skill 카드의 **SKILL.md** 보기를 누른다. 같은 파일은 GET 으로도 읽을 수 있다.

```
curl -s http://localhost:8910/agents/skills/hydraulic-cooling-response/files/SKILL.md | python3 -c "import sys,json;print(json.load(sys.stdin)['content'][:300])"
```

2. frontmatter 의 `name` 과 `tools` 를 찾는다.

```
name: hydraulic-cooling-response
description: 유압설비 냉각 이상·센서 오류 사건에 대해 근거 있는 조치 제안을 만드는 업무 지침 (교육용)
version: 1
tools: [get_asset_context, get_recent_window, search_sop, propose_action]
```

3. `agent_assignment.json` 의 `skills`, `mcp_servers`, `allowed_tools` 네 칸을 채운다. `allowed_tools` 는 frontmatter `tools` 와 같게 둔다.
4. 검사한 뒤, 감사 신호에 맞춰 할당을 등록한다.

```
PYTHONPATH=. .venv/bin/python -m pytest ../labs/integrated/S08 -q -k agent
curl -s -X POST http://localhost:8910/agents -H 'Content-Type: application/json' -d @../labs/integrated/S08/agent_assignment.json
```

확인 포인트

등록된 에이전트 기록(`GET /agents` , 2026-09-14):

```
[{"name": "유압설비 운영 에이전트", "role": "냉각 이상·센서 오류 사건에 대해 근거 있는 조치 제안을 만든다.", "model": "gpt-4.1-mini",
  "skills": ["hydraulic-cooling-response"], "agent_id": "ops-agent", "mcp_servers": ["hydops"],
  "allowed_tools": ["get_asset_context", "get_recent_window", "search_sop", "propose_action"],
  ...}]
```

콘솔 Agents 화면의 업무 에이전트 카드에 "할당 Skill: hydraulic-cooling-response"와 허용 도구 4개가 보이면 된다.

흔한 실수

- Skill 이름을 파일 경로(`skills/hydraulic-cooling-response/SKILL.md`)로 적는 것. 등록 이름은 frontmatter `name` 이다. 등록되지 않은 이름이면 `POST /agents` 가 404

`skill ... not registered` 로 거부한다.

- SKILL.md 에 SOP 절 번호를 박아 넣는 것. 실제 운영에서 "부하 감소 SOP §4 를 인용하라"고 적었더니 LLM 이 검색되지 않은 절을 인용했고, 근거 검증이 HOLD 로 막았다. Skill 에는 "검색 결과에 있는 절만 인용"을 쓴다.

4. 활동 2 — 제공 도구 목록에서 상태·검색·조치 도구 확인 · 50분

목표

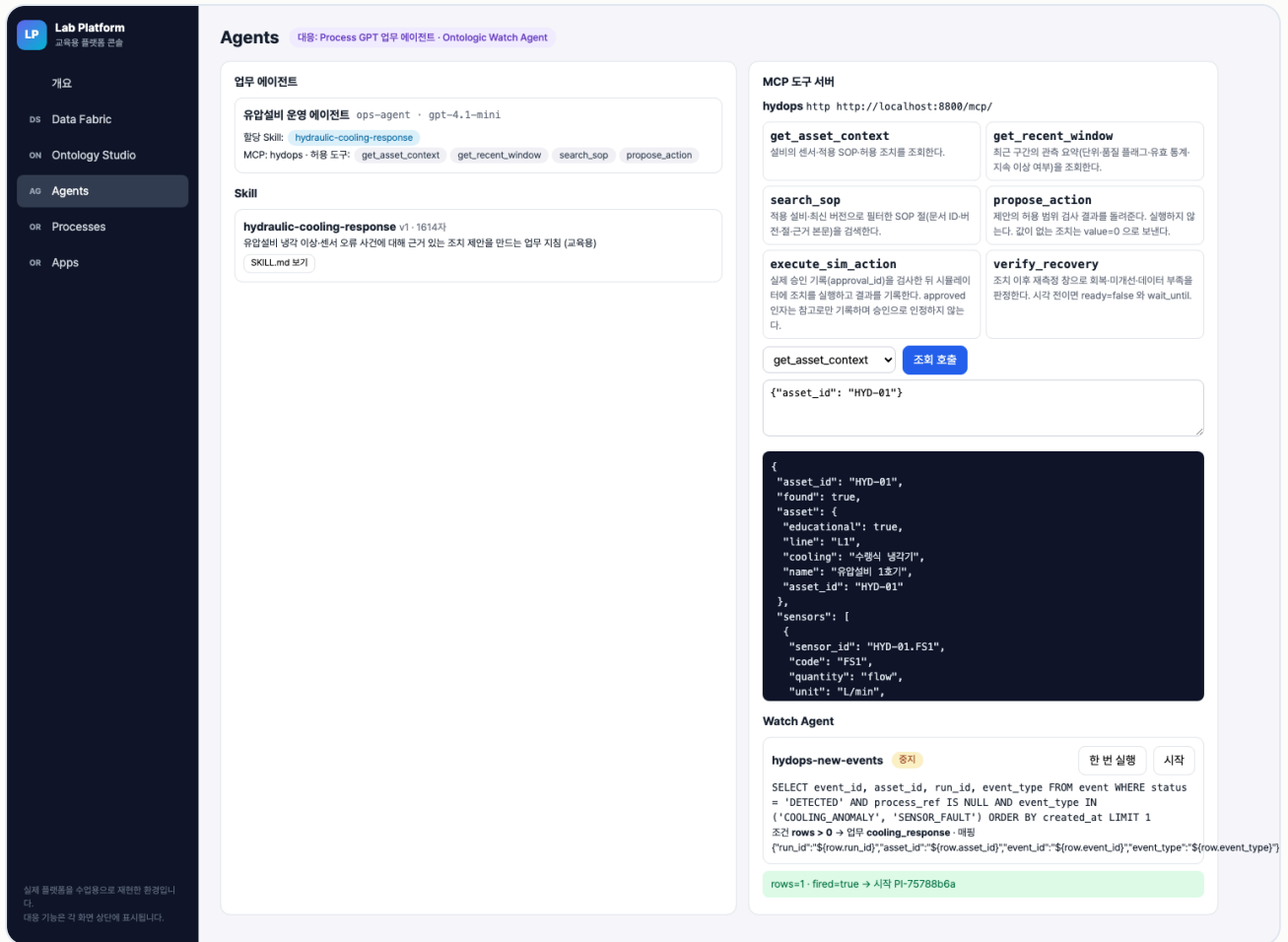
도구 서버의 도구 6종을 목록으로 받고, 역할별로 나눈 뒤 조회 도구 하나를 직접 호출한다.

따라 하기

1. 도구 목록을 받는다.

```
curl -s http://localhost:8910/agents/mcp-servers/hydops/tools | python3 -c  
"import sys,json;[print(t['name'],'|',list(t['args'])) for t in  
json.load(sys.stdin)]"
```

2. 콘솔 Agents 화면 오른쪽 **MCP 도구 서버** 에서 `get_asset_context` 를 고르고
`{"asset_id": "HYD-01"}` 로 **조회 호출** 을 누른다.
3. Watch Agent 카드의 SQL 과 매핑을 읽는다(시작 버튼은 강사만 누른다).



업무 에이전트·Skill·MCP 도구 조회 호출·Watch 실행

확인 포인트

도구 목록 (2026-09-14):

```
get_asset_context | ['asset_id']
get_recent_window | ['asset_id', 'seconds']
search_sop | ['asset_id', 'event_type', 'query']
propose_action | ['event_id', 'action_id', 'value', 'citations']
execute_sim_action | ['event_id', 'approval_id', 'action_id', 'value', 'approved']
verify_recovery | ['event_id', 'action_log_id', 'window_index']
```

분류	도구	업무 에이전트 허용	누가 부르나
상태 조회	get_asset_context, get_recent_window	허용	에이전트

분류	도구	업무 에이전트 허용	누가 부르나
근거 검색	search_sop	허용	에이전트
제안 검사(실행 안 함)	propose_action	허용	에이전트
조치 실행	execute_sim_action	불허	프로세스 도구 활동 "승인 조치 실행"
재측정	verify_recovery	불허	프로세스 도구 활동 "조치 후 재점검"

`execute_sim_action` 의 설명에는 "approved 인자는 참고로만 기록하며 승인으로 인정하지 않는다"고 적혀 있다. 코드(`hydops/b8_action/executor.py`)는 `approval_id` 로 승인 테이블을 찾아 없으면 `APPROVAL_NOT_FOUND` , 거부면 `APPROVAL_REJECTED` , 값이 다르면 `VALUE_MISMATCH` 로 거부한다.

Watch Agent 는 `SELECT event_id, asset_id, run_id, event_type FROM event WHERE status = 'DETECTED' AND process_ref IS NULL AND event_type IN ('COOLING_ANOMALY', 'SENSOR_FAULT') ORDER BY created_at LIMIT 1` 로 미처리 사건 **한 건**만 읽는다. S14 화면의 결과는 `rows=1 · fired=true` → 시작 `PI-75788b6a` 였다.

흔한 실수

- 도구 서버를 명령 기반 설정(`command: ...`)으로 등록하려는 것. 교육용 런타임은 `type: http` 와 URL 만 받는다.
- 실행 도구가 목록에 있으니 에이전트에게도 줘야 한다고 생각하는 것. 서버에 **있는 것**과 에이전트에 **허용하는 것**은 다르다. S08 테스트는 `allowed_tools` 에 실행 도구가 있으면 실패한다.
- 도구 인자 타입을 느슨하게 두는 것. 인용 인자를 `list[dict]` 로 두었더니 OpenAI 함수 스키마가 거부해 `Citation(doc_id, version, section)` 타입을 명시했다.

5. 활동 3 — 프로세스 템플릿에 이벤트 입력 매핑 · 50분

목표

Watch 가 읽은 사건 행이 접수 폼 → 에이전트 지시 → 승인 폼 → 실행 도구 인자로 이어지는 매핑을 `process_mapping.json` 에 채운다.

따라 하기

1. 템플릿의 활동 순서를 콘솔 **Processes** 상단에서 읽는다: 사건 접수(폼) → 사건-업무 대응 기록(시스템) → 근거 확인·조치 제안(에이전트) → 근거 검증·기록(시스템) → 조치 승인(사람) → 승인 기록(시스템) → 승인 조치 실행(도구) → 조치 후 재점검(도구).
2. 템플릿에서 승인 폼과 실행 도구 부분을 읽는다
(`system/labplatform/templates/process_cooling_response.json`).

```
{
  "id": "approve", "name": "조치 승인", "type": "human", "role": "설비 담당자",
  "fields": [
    {
      "name": "event_id", "label": "사건 ID", "type": "text", "readonly": true,
      "default": "${event_id}"
    },
    {
      "name": "proposed_value", "label": "제안값", "type": "number", "readonly": true,
      "default": "${proposed_value}"
    },
    {
      "name": "citations", "label": "근거 참조", "type": "citations", "readonly": true,
      "default": "${citations}"
    },
    {
      "name": "decision", "label": "결정", "type": "select", "options": ["APPROVED", "REJECTED"], "required": true
    },
    {
      "name": "approved_value", "label": "승인값", "type": "number", "default": "${proposed_value}", ...
    }
  ]
},
{
  "id": "act", "name": "승인 조치 실행", "type": "tool", "server": "hydops", "tool": "execute_sim_action",
  "args": {
    "event_id": "${event_id}", "approval_id": "${approval_id}",
    "action_id": "${action_id}", "value": "${approved_value}"
  }
}
```

3. `process_mapping.json` 의 빈칸을 채운다. `${row....}` 는 Watch SQL 결과 열, `${...}` 는 인스턴스 변수다.
4. 검사한다. 테스트는 플랫폼 엔진의 실제 치환 함수 `labplatform.process.render` 로 매핑을 채워 보고, 이미 완료된 인스턴스의 승인 폼 값과 같은지도 대조한다.

```
PYTHONPATH=. .venv/bin/python -m pytest ../labs/integrated/S08 -q
```

5. 정의가 등록돼 있는지 확인한다(강사 준비 단계 `process` 에서 등록한다).

```
curl -s http://localhost:8910/process/definitions | python3 -c "import sys,json;d=json.load(sys.stdin)[0];print(d['def_id'],len(d['activities']))"
```

확인 포인트

- 정의 확인 출력: `cooling_response 13` (활동 8개 + 종료 5개: 종결·이관·재측정 보류·미실행 종료·보류)
- 실제 인스턴스 `PI-75788b6a` 에 남은 값(2026-09-14):

단계	기록된 입력
사건 접수	<code>event_id EVT-20260914-131041-HYD-01-f81e</code> , <code>asset_id HYD-01</code> , <code>run_id RUN-b5021e42b2</code> , <code>event_type COOLING_ANOMALY</code> , <code>source watch:hydops-new-events</code>
근거 확인·조치 제안	지시문 "사건 EVT-20260914-131041-HYD-01-f81e 을 처리하라. - asset_id: HYD-01 ..."
조치 승인 폼	<code>event_id·asset_id·run_id</code> 같음, <code>action_id REDUCE_LOAD</code> , <code>proposed_value 0.8</code> , 근거 <code>SOP-COOL-001 v2 §4</code> , <code>SOP-LOAD-001 v1 §4</code>
승인 조치 실행	<code>execute_sim_action</code> 인자 <code>event_id</code> 같음, <code>approval_id APR-f5b6f889a6</code> , <code>value 0.8</code>

인스턴스 키는 `cooling_response:EVT-20260914-131041-HYD-01-f81e:1` 이다. 같은 사건으로 다시 시작하면 새 인스턴스를 만들지 않고 `{"duplicate": true, "instance_id": ...}` 로 기존 인스턴스를 돌려준다(`out/e2e_platform_report.json` 정상 회복 사례).

흔한 실수

- 실행 도구의 `value` 를 `${proposed_value}` 로 매핑하는 것. 사람이 승인 폼에서 값을 바꿀 수 있으므로 실행은 `${approved_value}` 로 한다. 코드는 승인값과 다른 실행값을 `VALUE_MISMATCH` 로 거부한다.
- 인스턴스 키를 `asset_id` 로 두는 것. 같은 설비에서 다음 사건이 나면 중복으로 막힌다. 키는 `event_id` 다.
- 시작 요청을 여러 번 보내면 중복이 알아서 막힌다고 가정하는 것. 이 환경은 키(정의: `event_id:회차`)를 명시적으로 등록해서 막는다.

6. 활동 4 — 사람 승인과 거부 결과를 업무 화면에서 확인 · 50분

목표

워크리스트의 조치 승인 태스크를 승인·거부했을 때 인스턴스 단계와 사건 기록이 어떻게 달라지는지 비교한다.

따라 하기

1. 강사가 관제 화면(:8800)에서 HYD-01 에 "냉각 성능 저하"를 주입하고, Agents 화면 Watch 카드의 **한 번 실행** 으로 업무를 시작한다.
2. **Processes** → 워크리스트에 "조치 승인 · 설비 담당자" 태스크가 뜨면 폼의 읽기 전용 칸을 먼저 읽는다.

Lab Platform
교육용 플랫폼 운영

개요
DS Data Fabric
ON Ontology Studio
AG Agents
OR **Processes**
OR Apps

Processes 대응: Process GPT 프로세스 · 워크리스트

냉각 이상 대응 cooling_response v1

사건 접수 폼 | 사건-업무 대응 기록 시스템 | 근거 확인-조치 제안 에이전트 | 근거 검증-기록 시스템 | **조치 승인 사람** | 승인 기록 시스템 | 승인 조치 실행 도구 | 조치 후 재점검 도구

워크리스트 사람 태스크

조치 승인 설비 담당자 EVT-20260914-131041-HYD-01-f81e

사건 ID: EVT-20260914-131041-HYD-01-f81e | 설비 ID: HYD-01
실행 ID: RUN-b5021e42b2 | 제안 조치: REDUCE_LOAD
제안값: 0.8 | 근거 참조: SOP-COOL-001 v2 §4, SOP-LOAD-001 v1 §4

제안 이유: 센서 TS1의 상태가 정상이며 센서 오류가 없으므로 설비 조치가 가능합니다. 적용 SOP에서 냉각 이상에 대해 승인 부하 감소 조치를 허용하고 있습니다. 제안 한 부하 감소 값 0.8은 SOP 허용 범위 내이며, 조치 제안 검사를 통과하였습니다.

결정: **APPROVED**

승인값: 0.8 | 승인자: kim.operator

의견: [Text Area]

제출 승인 기록이 조치 도구의 실행 조건이 됩니다.

업무 인스턴스

RUNNING 조치 승인 WAITING_HUMAN
PI-75788b6a · EVT-20260914-131041-HYD-01-f81e

실제 플랫폼을 수업용으로 제한한 환경입니다.
대응 가능한 각 화면 상단에 표시합니다.

조치 승인 태스크 폼

3. 에이전트-조치 담당이 결정 **APPROVED** , 승인값 **0.8** , 승인자 이름을 넣고 **제출** 한다(다음 사건에서는 역할을 바꿔 **REJECTED** 로 제출한다).
4. 인스턴스를 열어 단계 기록을 본다.

LP

Lab Platform

교육용 플랫폼 콘솔

개요

DSData Fabric

ONOntology Studio

AGAgents

ORProcesses

ORApps

현재 플랫폼을 수업용으로 제한한 환경입니다.

대용 기능은 각 화면 상단에 표시됩니다.

Processes

대용: Process GPT 프로세스 - 워크리스트

냉각 이상 대응cooling_response v1

사건 접수

사건-업무 대응 기록

근거 확인-조치 제안

근거 검증-기록

조치 승인

승인 기록

승인 조치 실행

조치 후 재점검

워크리스트

사람 태스크

대기 중인 태스크가 없습니다.

업무 인스턴스

COMPLETED

종결 (재측정 회복)

DONE

PI-75788b6a · EVT-20260914-131041-HYD-01-f81e · 설비 RECOVERED

냉각 이상 대응 · PI-75788b6a

COMPLETED

업무 결과: 종결

설비 재측정: RECOVERED

인스턴스 키cooling_response:EVT-20260914-131041-HYD-01-f81e:1 — 업무 완료 상태는 설비 회복을 대체하지 않습니다.

1

사건 접수

종

DONE

13:10:19

▶ 입력-출력

2

사건-업무 대응 기록

시스템

DONE

13:10:19

▶ 입력-출력

3

근거 확인-조치 제안

에이전트

DONE

13:10:19

PROPOSE REDUCE_LOAD 0.8 · 9.9s · 도구 · get_recent_window · get_asset_context · search_sop · propose_action

▶ 입력-출력

4

근거 검증-기록

시스템

DONE

13:10:29

▶ 입력-출력

5

조치 승인

사람

DONE

13:10:29

▶ 입력-출력

6

승인 기록

시스템

DONE

13:10:32

▶ 입력-출력

7

승인 조치 실행

도구

DONE

13:10:32

▶ 입력-출력

8

조치 후 재점검

도구

DONE

13:10:32

▶ 입력-출력

9

종결 (재측정 회복)

종료

DONE

13:10:50

▶ 입력-출력

인스턴스 단계 기록 — 업무 결과 종결·설비 재측정 RECOVERED

확인 포인트

out/e2e_platform_report.json (2026-09-14 실행)의 두 사례를 비교한다.

항목	승인(정상 회복)	거부
마지막 단계	조치 후 재점검 → 종결 (재측정 회복)	승인 기록 → 미실행 종료
인스턴스 business_result	종결	승인 거부
인스턴스 equipment_outcome	RECOVERED	null
사건 status	CLOSED	REJECTED
승인 기록	kim.operator · APPROVED · 0.8	kim.operator · REJECTED · null

항목	승인(정상 회복)	거부
실행 기록(<code>actions</code>)	SUCCEEDED 1건	0건
재측정(<code>verifications</code>)	RECOVERED 1건	0건

같은 승인 태스크를 한 번 더 제출하면 HTTP **409** "이미 처리됐거나 대기 중인 태스크가 아니다"로 거부된다(보고서 `second_completion_http: 409`).

S16 화면의 PI-75788b6a 는 9단계가 모두 DONE 이고, 상단에 "업무 결과: 종결"과 "설비 재측정: RECOVERED" 가 **따로** 표시된다.

흔한 실수

- 거부했는데 사건이 "끝났으니 회복"이라고 적는 것. 거부는 미실행 종료이고 설비 상태는 판정하지 않았다(`equipment_outcome null`).
- 인스턴스가 FAILED 로 멈췄을 때 다시 시작 버튼만 누르는 것. 실제 운영에서 플랫폼 프로세스가 API 키를 못 읽어 에이전트 단계가 ERROR 가 되었고, 인스턴스는 FAILED, 사건은 DETECTED 에 머물렀다. 실패가 기록으로 남는 것 자체가 확인 포인트다. 단계의 `error` 를 읽고 강사에게 알린다.

7. 실습 과제

- 폴더: `labs/integrated/S08/` (`TASK.md` 참고)
- 빈칸 위치
 - `agent_assignment.json`: `skills` 1칸, `mcp_servers` 1칸, `allowed_tools` 4칸
 - `process_mapping.json`: 정의 ID, 인스턴스 키, Watch 행 매핑 4칸, 접수 필수 필드 2칸, 승인 폼 역할·기본값 6칸·결정 선택지 2칸·승인값 기본값, 승인 기록 값, 실행 도구 이름·인자 2칸, 실행으로 가는 결정값
- 검증 방법: SKILL.md frontmatter·제공 템플릿과 같은지(정적), 실행 도구 제외, `render()` 결과, 등록된 Skill·도구·정의와 대조(GET), 완료 인스턴스의 승인 폼 값과 대조(GET). 테스트는 프로세스를 시작하지 않는다.
- 확인 명령: `cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest ../labs/integrated/S08 -q` (정답본 9 passed)

- 경험자 과제: 승인 폼에 읽기 전용 필드를 하나 더 두려면 템플릿 어느 두 곳(에이전트 `outputs` , 승인 `fields`)을 바꿔야 하는지 적는다.

8. 완료 확인 체크리스트

- ☐ 업무 에이전트 기록에 할당 Skill 1개와 허용 도구 4개가 있고 실행 도구가 없다.
- ☐ 도구 6종을 상태·검색·제안 검사·실행·재측정으로 나눈 표를 냈다.
- ☐ `process_mapping.json` 테스트가 통과했다.
- ☐ 한 인스턴스에서 접수 폼·승인 폼·실행 도구 인자의 `event_id` 가 같음을 확인했다.
- ☐ 승인 사례와 거부 사례의 `business_result` · `equipment_outcome` · 실행 기록 수를 비교했다.
- ☐ 새 에이전트 코드를 작성하지 않고 JSON 설정만으로 연결했다.

9. 회고 (10분)

1. 에이전트에게 실행 도구를 주지 않아도 조치가 실행되는 경로는 어디인가?
2. 승인값을 제안값과 다르게 넣었을 때 어떤 기록이 남을지 예상해 본다.
3. "프로세스 완료"와 "설비 회복"을 구분하지 않으면 관제 화면에서 어떤 오판이 생기나?

10. 실제 플랫폼 대응

교육용 플랫폼 기능	실제 플랫폼 기능	다른 점
<code>POST /agents/skills</code> (SKILL.md 등록), <code>POST /agents</code> (할당)	Process GPT 업무 에이전트 Skill 할당	교육용은 짧은 SKILL.md 한 파일만 받고, 본문을 최대 6,000자까지 시스템 프롬프트에 붙인다
<code>POST /agents/mcp-servers</code> (<code>type: http</code>), <code>GET .../tools</code> , <code>POST .../call</code>	Process GPT 에이전트 도구 (MCP) 연결	교육용은 streamable HTTP 한 방식만 받는다

교육용 플랫폼 기능	실제 플랫폼 기능	다른 점
<code>allowed_tools</code> 필터	에이전트 도구 권한 설정	교육용은 이름 목록 비교 한 줄로 단순화했다
<code>POST /process/definitions</code> , 인스턴스 키 <code>def:event_id:회차</code>	Process GPT 프로세스 정의·인스턴스	교육용 활동 종류는 form·http·agent·human·tool·end 여섯 가지다
<code>GET /process/worklist</code> , <code>POST /process/workitems/{id}/complete</code>	Process GPT 워크리스트·사람 태스크	교육용은 역할 이름만 비교하고 계정 권한은 두지 않는다
<code>POST /agents/watch</code> , <code>.../tick</code>	Ontologic Watch Agent	교육용은 <code>rows > N</code> 조건 하나, 한 번에 한 행만 처리한다
아이나루	(9회에서 다룬다)	—

강사 노트

시간 배분

시각	내용
09:00~09:50	활동 1 — Skill 읽기, 할당 JSON 채우기, <code>POST /agents</code>
09:50~10:00	휴식
10:00~10:50	활동 2 — 도구 목록 분류, 조회 호출, Watch SQL 읽기
10:50~11:00	휴식
11:00~11:50	활동 3 — 입력 매핑 빈칸 (역할 교대: 관계·문서 담당 ↔ 에이전트·조치 담당)
11:50~12:00	휴식
12:00~12:50	활동 4 — 승인 사례 1건, 거부 사례 1건

시각	내용
12:50~13:00	회고

사전 준비

- 서버 `scripts/servers.sh platform 4` (사건 처리를 교육용 플랫폼이 맡는 모드).
- `scripts/bootstrap_platform.py --upto watch`. 단계는 누적 실행이라 데이터소스·스키마 가져오기·발행·Skill·도구 서버·에이전트·프로세스·Watch 가 모두 제공 템플릿 값으로 다시 등록된다(발행은 v1 부터 다시 매겨진다). Watch 는 `enabled: false` 로 등록되므로 활동 4 에서 **한 번 실행** 으로만 시작한다.
- OpenAI 키가 플랫폼 프로세스에서 읽히는지 리허설한다. 키를 못 읽으면 에이전트 단계가 ERROR 로 끝난다.
- 사건이 여러 팀에 동시에 필요하면 강사가 순서대로 주입한다. 설비·유형당 열린 사건은 하나라서, 앞 사건이 끝나기 전에는 같은 설비에 새 냉각 이상 사건이 생기지 않는다.

장애 대응 플랫폼이 멈추면 6회 로컬 흐름(`scripts/servers.sh local 4` , 관제 화면 승인 버튼)으로 승인·거부 비교를 이어 간다. 플랫폼 실습 완료로 기록하지 않고, 복구 후 활동 3·4 를 보충한다.

용어

용어	뜻
업무 에이전트	할당 Skill 과 허용 도구로 근거 확인·제안만 하는 에이전트. 승인·실행 권한이 없다
Skill 할당	등록된 SKILL.md 를 에이전트 설정 <code>skills</code> 에 넣는 일
MCP 도구 서버	도구 6종을 제공하는 서버 <code>http://localhost:8800/mcp/</code>
허용 도구(allowed_tools)	에이전트가 실제로 받는 도구 이름 목록
인스턴스 키	<code>정의:event_id:회차</code> . 같은 사건의 중복 시작을 막는다
워크리스트	사람 태스크 목록. 조치 승인 태스크가 여기서 기다린다
business_result / equipment_outcome	업무 결과(종결·이관·승인 거부·보류) / 재측정으로 판정한 설비 결과